MIP Formulations for Delete-Free AI Planning

Salvagnin Domenico* Zanella Matteo†

Abstract

We investigate existing Mixed Integer Programming (MIP) formulations for cost-optimal delete-free STRIPS planning: these models are built by enforcing acyclicity in the underlying causal relation graph using time labeling and vertex elimination methods. We then propose two new approaches to modeling acyclicity, one based on subtour elimination constraints and the other based on disjunctive landmark constraints. In addition, we propose to warm-start the models with the landmarks computed by the LM-cut heuristic, and describe a simple greedy primal heuristic to provide a starting feasible solution to the MIP solver. Our results demonstrate that the proposed techniques outperform the current state of the art.

1 Introduction

AI planning is a field of artificial intelligence that deals with *planning problems*, i.e., problems where we look for a cost-optimal sequence of actions (a *plan*) to reach a desired goal state from some initial state. In its classical version, the current state is described by a finite set of boolean variables, called *facts*, and each action is characterized by its cost, by its prerequisites (a subset of the facts that need to be true for the action to be applicable), and by its effects, i.e., which facts become true (*add* effects) and which become false (*delete* effects) as the result of applying the action. Solving a planning problem is in general very challenging, as both its feasibility and optimality versions are known to be PSPACE-complete (Bylander (1994)).

The connection between classical AI planning and mathematical optimization methods have been the subject of active research in the last decade (and more), ranging from MIP formulations for planning problems (or their relaxations) (van den Briel et al. (2008), Imai and Fukunaga (2015), Piacentini et al. (2018), Feyzbakhsh Rankooh and Rintanen (2022)), to application of linear programming techniques for performing optimal cost partitioning (Pommerening (2017), Pommerening et al. (2019, 2021)), and to decomposition methods akin to logic-based Benders decomposition (Davies et al. (2016)). Even more recently, the connection between classical planning and MDPs has been the subject of several studies, see for example Klößner et al. (2022): this is still at the early stages of investigation though, and MDP techniques have not been empirically evaluated on classical planning benchmarks yet. We finally note that there are also connections with dynamic programming and several techniques used therein: for example, the so-called state-space relaxation (Christofides et al. (1981)), extensively used in column generation approaches for routing applications, has a planning analogous in the CEGAR technique (Seipp and Helmert (2018)), that iteratively refines a planning relaxation.

The *delete-free relaxation* of a planning problem is a relaxation where for each action we drop its delete effects: in other words, in a delete-free planning problem, once a fact becomes true it can never become false again. This property has far reaching consequences: each action can be applied at most once, as there is no benefit in using an action multiple times, feasibility can be checked in polynomial time, and the length of

*Department of Information Engineering, University of Padova, Email: salvagni@dei.unipd.it

†Department of Information Engineering, University of Padova, Email: matteo.zanella.3@phd.unipd.it

any optimal plan becomes polynomial in the number of actions and facts. At the same time, computing h^+, that is, the cost of the optimal delete-free plan, is NP-hard (Bylander (1994)).

The *delete-free relaxation* has been the subject of extensive research, for several reasons. First of all, being a relaxation, h^+ provides an *admissible heuristic*, i.e., a valid dual bound, for A^* search, which is one of the state of the art approaches for solving general planning problems. Second, it provides a reference for other common dual bounding functions, such as $h^{\max}$ (Bonet and Geffner (2001)) and *LM-cut* (Helmert and Domshlak (2009)), that are valid for the delete-free relaxation of a planning problem, and thus dominated by h^+. Another reason for the importance of delete-free planning is that optimal plans for general planning problems can be computed by iteratively solving and reformulating delete-relaxed problems (Haslum (2012)). Finally, some combinatorial problems, like the minimal seed-set problem (Gefen and Brafman (2011)) and determining join orders in relational database query plan generation (Robinson et al. (2014)), can be naturally modeled as delete-free problems, so delete-free planning is interesting in its own right.

The current state of the art for solving a delete-free planning problem consists in formulating it as a combinatorial optimization problem, i.e., a declarative set of variables and constraints, and using an off-the-shelf solver (SAT, CSP or MIP, depending on the chosen paradigm). In this paper, we consider MIP formulations of delete-free planning problems.

The contributions of this paper can be summarized as follows:

- we propose two new MIP formulations for delete-free planning, one based on subtour elimination constraints, and one based on disjunctive action landmarks;
- we propose two warm-starting techniques, applicable to all the formulations considered, old and new: a greedy primal heuristic providing a MIP start, and the initialization of the model with the landmark constraints computed by LM-cut;
- we show that subtour elimination constraints can be separated in polynomial time over both integer and fractional solutions, and describe the corresponding separation algorithm;
- we prove that separating landmark constraints over fractional solutions is instead NP-hard, and we provide both an exact separator, based on a MIP formulation, and a polynomial separation heuristic;
- we provide an extensive computational evaluation on the 2799 instances of the standard IPC benchmarks, showing that our best method significantly improves upon the current state of the art.

In Section 3, we review the two MIP formulations known from the literature, namely the time-stamp formulation by Imai and Fukunaga (2015) and the formulation based on vertex elimination by Feyzbakhsh Rankooh and Rintanen (2022), which is the current state of the art. Those two formulations share a common base structure, but differ in the way they model acyclicity, i.e., the fact that we cannot have a circular dependency between actions in any plan. The former uses a more compact encoding based on time-stamps, that results in smaller problems but also weaker linear programming (LP) relaxations. The latter is based on a vertex elimination order of the *causal relation graph*: this requires a number of constraints that is, in the worst case, cubic w.r.t. the number of facts in the problem, and it is thus quite expensive as a MIP encoding, in particular on large-scale problems, yet it provides provably stronger LP relaxations, and thus eventually proves to be the most effective method overall.

Both the formulations by Imai and Fukunaga (2015) and Feyzbakhsh Rankooh and Rintanen (2022) have polynomial size, and can thus be solved by just feeding them into a black-box MIP solver. Inspired by the connection to the *traveling salesman problem* (TSP), where we also have to model acyclicity, but the current state of the art is *not* based on polynomial formulations, in Section 4 we introduce two new models with an exponential number of constraints, thus requiring the dynamic separation of so-called violated *lazy constraints* on the fly during the solution process. The first model is based on *subtour elimination*

constraints (Dantzig et al. (1954)), again inspired by the TSP, while the second is based on the formulation of a delete-free planning problem as a *hitting set* problem over a set of (exponentially many) *disjunctive landmark* constraints, an approach dating back to Bonet and Castillo (2011) and Haslum et al. (2012).

In Section 5, we investigate how to improve the performance of the formulations described, both old and new. In detail, we noticed empirically that the MIP solver sometimes struggled in finding a good quality feasible solution early on in the search, which is unusual and quite surprising, given that it is always trivial to find a feasible solution in the delete-free case: we thus designed a simple greedy heuristic based on h^{add} (Bonet and Geffner (2001)), and used the solution obtained in this way as a MIP start for the solver. We also noticed that in some problems the LP relaxation of the MIP model was significantly weaker than what could be obtained much faster by the well-known LM-cut heuristic (Helmert and Domshlak (2009)), which is also based on disjunctive landmarks. We thus decided to warm-start each formulation with the set of landmarks computed by LM-cut itself.

Separating subtour elimination constraints (SEC) and/or disjunctive landmarks needs to be done in our branch-and-cut approach each time an integer solution is encountered, in order to guarantee the correctness of the method. However, the current state of the art for many lazy constraint approaches, like TSP (Applegate et al. (2007)) and Benders (Bonami et al. (2020)), does not just separate constraints at integer solutions: to the contrary, it usually deploys a full branch-and-cut approach, where cutting planes are separated also at fractional nodes. In Section 6, we thus investigate how to separate SEC and disjunctive landmarks on fractional solutions, showing that the former can still be done in polynomial time, while the latter is NP-hard. We provide nevertheless an exact approach based on a MIP formulation and a polynomial separation heuristic.

In Section 7 we provide an extensive computation evaluation of the proposed formulations, comparing them against the previous state of the art, and measuring the impact of the different enhancements, showing that some of the proposed methods significantly improve upon current state of the art. Conclusions and potential lines of future work are finally drawn in Section 8.

2 Background and notation

A delete-free *STRIPS* (Fikes and Nilsson (1971)) *planning problem* is a 5-tuple $\Pi = (P, A, I, G, \text{cost})$: P is a finite set of boolean variables (or *facts*); A is a finite set of *actions*: each action $a \in A$ is a pair $(\text{pre}(a), \text{add}(a))$, where $\text{pre}(a), \text{add}(a) \subseteq P$ denote respectively the set of *preconditions* and *add effects* of a; $I, G \subseteq P$ are, respectively, the *initial* and *goal* state; $\text{cost} : A \to \mathbb{R}_0^+$ is a cost function that maps each action to a non-negative real number. Executing an action a from a state $S \subseteq P$ generates a new state $S' = S \cup \text{add}(a)$. Let $\pi = [a_{\pi_1}, ..., a_{\pi_n}]$ be a finite sequence of n actions: we denote with $S(\pi) = I \cup (\bigcup_{a \in \pi} \text{add}(a))$ the state potentially reachable through π[1]. A *solution* is a sequence of actions for which $G \subseteq S(\pi)$. Moreover, a *feasible solution*, or *plan*, is a solution that satisfies $\text{pre}(a_{\pi_i}) \subseteq S([a_{\pi_1}, ..., a_{\pi_{i-1}}])$ for all $i \in \{1, \dots, n\}$, meaning that the preconditions of each action are met before it is executed. Finally, the cost of an action sequence π is $\text{cost}(\pi) = \sum_{a \in \pi} \text{cost}(a)$, and our goal is to find an *optimal plan* π^* with minimum cost h^+. Without loss of generality, we can assume the initial state I to contain a single fact: if this is not the case, we can simply add a dummy initial fact s, $P = P \cup \{s\}$, and a dummy action a_s, $A = A \cup \{a_s = (\{s\}, I)\}$, with $\text{cost}(a_s) = 0$. Analogously, we can also assume the goal state G to contain a single fact: if not, add a dummy goal fact t, $P = P \cup \{t\}$, and the dummy action a_t, $A = A \cup \{a_t = (G, \{t\})\}$, with cost $\text{cost}(a_t) = 0$. It is easy to see that any optimal plan π^* for the resulting problem is composed of an original problem's optimal plan π'^*, guarded by a_s and a_t, $\pi^* = (a_s, \pi'^*, a_t)$: the optimal cost is then unchanged. In the rest of the paper, we will thus always assume that $I = \{s\}$ and $G = \{t\}$.

[1] The definition of $S(\cdot)$ can be naturally extended to sets.

Throughout this work, we will use these notations:

$$A_{p\rightarrow} \coloneqq \{a \in A \mid p \in \text{pre}(a)\} \quad A_{\rightarrow q} \coloneqq \{a \in A \mid q \in \text{add}(a)\} \quad A_{p\rightarrow q} \coloneqq \{a \in A \mid p \in \text{pre}(a), q \in \text{add}(a)\}$$

2.1 $\mathbf{h}^{\max}$ and $\mathbf{h}^{\text{add}}$

The $h^{\max}$ function is a further relaxation of a planning problem in which, for each action, only the most expensive precondition to reach is considered. Given a state $S \subseteq P$ and a fact $q \in P$, it is formally defined by the recursive relation:

$$h_S^{\max}(q) = \begin{cases} 0 & \text{if } q \in S \\ \min_{a \in A_{\rightarrow q}} \{\text{cost}(a) + \max_{p \in \text{pre}(a)} \{h_S^{\max}(p)\}\} & \text{otherwise} \end{cases} \tag{1}$$

Once $h_S^{\max}(q)$ is computed for every $q \in P$, the $h^{\max}$ value of the state S itself is defined as $h_S^{\max} = h_S^{\max}(t)$, the relaxed cost of reaching t from S. We also define $h^{\max} = h_{\{s\}}^{\max}$. Due to $h^{\max}$ being a relaxation of the problem Π, the value of $h^{\max}$ is a lower bound on h^+. h^{add} function's definition is identical to that of $h^{\max}$, except for using the sum of each precondition's cost instead of the maximum in Equation 1. Both functions date back to Bonet and Geffner (2001) and can be computed in polynomial time by a generalized Dijkstra algorithm.

2.2 Landmarks

A *disjunctive action landmark* $L \subseteq A$ for Π (*landmark* for short) is a set of actions such that every plan for Π must contain at least one action in L. A key result (Bonet and Helmert (2010)) is that the computation of the optimal plan for a delete-relaxed planning problem can be formulated as a minimum-cost hitting set problem over all of its landmarks.

2.3 Justification Graphs

A *justification graph* (Helmert and Domshlak (2009)) is a directed graph $G_D = (P, E_D)$, that has the set of facts as nodes, and whose edges are defined by a *precondition choice function* (*pcf* for short) $D : A \rightarrow P$ so that $(p, q) \in E_D$ if and only if there is an action a with $D(a) = p \in \text{pre}(a)$ and $q \in \text{add}(a)$. For a given justification graph G_D, an *s-t-cut* is a partition $(S, P \setminus S)$ such that $s \in S$ and $t \notin S$, and its *cut-set* is the set of actions crossing the cut in the forward direction, i.e., with $D(a) \in S$ and $\text{add}(a) \nsubseteq S$. It is easy to show that, since any plan defines an s-t-path in G_D, the actions in any cut-set define a valid landmark for Π.

2.4 LM-cut

LM-cut (Helmert and Domshlak (2009)) is a state of the art admissible heuristic for classical AI planning, based on the iterative computation of disjunctive landmarks: at each step of the process, a pcf based on the $h^{\max}$ values of the facts is chosen, an s-t cut-set is identified yielding a landmark L, and the cost of actions in L is reduced by the minimum between their costs. The process is repeated until no cut-set of positive (reduced) cost can be found. The value h^{LM} computed by LM-cut is valid for the delete-free relaxation of a planning problem, and thus is a lower bound on h^+. We refer to Helmert and Domshlak (2009), Lauer and Fickert (2020) for details about the procedure.

It is worth noting that the LM-cut procedure is not completely specified, as in general there are degrees of freedom in the choice of the pcf: for a given action a, we might have multiple preconditions achieving the maximum value. In addition, different pcfs can give significantly different lower bounds (Lauer and Fickert (2020)).

3 Compact MIP formulations

We now define the basic MIP formulation for a delete-free problem. At a bare minimum, the model needs to keep track of which actions are used in a solution: since in a delete-free problem an action can be used at most once in a plan π, this naturally translates into a set of binary variables $x_a, \forall a \in A$, that take value 1 if and only if $a \in \pi$. In general, we also need to keep track of which facts become true, in order to enforce that the goal is reached and that the preconditions of actions are met, and this translates into another set of binary variables $y_p, \forall p \in P$, that take value 1 if and only if $p \in S(\pi)$. Finally, we need to keep track of which action is responsible for making a fact true for the first time, i.e., which action $a \in A$ is the first achiever of a fact $q \in P$, so we have binary variables $z_{a,q}$ that take value 1 if and only if a is the *first achiever* of q in π. Of course, $z_{a,q}$ is defined only for pairs (a, q) for which $q \in \text{add}(a)$.

With this initial set of variables, a first (incomplete) model, first introduced by Imai and Fukunaga (2015) and then refined by Feyzbakhsh Rankooh and Rintanen (2022), reads:

$$\min \sum_{a \in A} \text{cost}(a) x_a \tag{2}$$

$$\sum_{a \in A_{\to q}} z_{a,q} = y_q \qquad \forall q \in P \setminus \{s\} \tag{3}$$

$$\sum_{a \in A_{p \to q}} z_{a,q} \leq y_p \qquad \forall p, q \in P \tag{4}$$

$$z_{a,q} \leq x_a \qquad \forall a \in A, \forall q \in \text{add}(a) \tag{5}$$

$$y_s, y_t = 1 \tag{6}$$

$$x_a, z_{a,q}, y_p \in \{0, 1\} \qquad \forall a \in A, \forall q \in \text{add}(a), \forall p \in P \tag{7}$$

The objective function (2) just sums up the costs of the actions used. As for the constraints, (3) encodes that a fact is achieved if and only if an action is its first achiever and that there can be at most one first achiever per fact, (4) that an action can be a first achiever only if its preconditions are achieved, (5) that an action can be a first achiever only if it is used and (6) that the initial fact is achieved by default, and the goal fact must be achieved.

We denote with IP(Π) the model described by constraints (2) to (7), and with $\hat{x}$ any of its integer feasible solutions.

As anticipated, unfortunately this model is incomplete, as it fails to prevent circular dependencies between actions: for example, we can select an action a with prerequisite p and add-effect q, together with an action b with prerequisite q and add-effect p, without any other selected action. In order to obtain a complete (and correct) model, we thus need to model causal *acyclicity* between the actions in the solution.

3.1 Modeling acyclicity

A general framework for reasoning about acyclicity is the so-called *causal relation graph* (Feyzbakhsh Rankooh and Rintanen (2022)). Given a delete-relaxed problem Π, its causal relation graph is defined as the directed graph $G_\Pi = (P, E_\Pi)$, where an arc $(p, q) \in E_\Pi$ if and only if $A_{p \to q} \neq \emptyset$.

Any integer feasible solution $\hat{x}$ of IP(Π) induces a subgraph $G_{\hat{x}} = (P, E_{\hat{x}})$ of G_Π, where we select only the edges (p, q) where q is first achieved by some action a, and thus $\hat{z}_{a,q} = 1$, and p are all the preconditions of such action. As shown in (Feyzbakhsh Rankooh and Rintanen (2022)), for $\hat{x}$ to be associated with a plan, $G_{\hat{x}}$ needs to be acyclic: a cycle in this graph constitutes a cyclical causal dependency between two or more facts, which means none of these facts can be applied before the others. The existing formulations in the literature differ in how cycles are forbidden in $G_{\hat{x}}$.

3.2 Time labeling

The first approach to deal with acyclicity is due to Imai and Fukunaga (2015), and further refined by Feyzbakhsh Rankooh and Rintanen (2022). The basic idea is to simply assign a time label to each fact and to ensure that, whenever an action a is the first achiever of a fact q, the time stamp assigned to q must be greater by at least 1 of the timestamp of any precondition p of a. Timestamps are encoded with integer variables $t_p \in \{1, ..., |P|\}$ for any $p \in P$, and the model is completed with the constraints:

$$t_p - t_q + 1 \leq |P|(1 - z_{a,q}) \qquad \forall p, q \in P, \forall a \in A_{p \to q} \tag{8}$$

We denote the model that uses Time Labeling as TL(Π). Note that in the constraints (8), the number of facts $|P|$ acts as a big-M coefficient: as such, we do not expect the model TL(Π) to have a strong LP relaxation. On the other hand, the model is quite compact, requiring only $|P|$ additional variables and $O(|A| \cdot |P|^2)$ additional constraints.

3.3 Vertex elimination

A completely different approach is based on vertex elimination graphs, a concept originally introduced by Rose et al. (1976): the idea is to introduce variables that encode precedence between two facts and then enforce acyclicity with those variables. Given a directed graph $G = (P, E)$, let $O = [p_1, ..., p_{|P|}]$ be an arbitrary ordering of the vertices P. According to the ordering O, we construct a sequence of graphs $G_0 = G, ..., G_{|P|}$ by eliminating the vertices of G, where $G_i = (P_i, E_i)$ is produced from G_{i-1} by eliminating p_i, and adding edges from all its in-neighbors to all its out-neighbors. Given an ordering[2] O, we can apply the vertex elimination procedure to the causal relation graph G_Π, and obtain the graph $G^* = (P, E^*_\Pi = \cup_{i=0}^{|P|} E_i)$; let Δ be the set of all ordered triplets (p_i, p_j, p_k) encoding the set of edges (p_i, p_k) added when eliminating p_j in the vertex elimination process. We can enforce $G_{\hat{x}}$ to be acyclic by adding the binary variables $e_{p,q}$, which will take value of 1 if and only if p precedes q, for any edge $(p, q) \in E^*_\Pi$, together with the constraints:

$$z_{a,q} \leq e_{p,q} \quad \forall p, q \in P, \forall a \in A_{p \to q} \tag{9}$$

$$e_{p,q} + e_{q,p} \leq 1 \quad \forall (p, q) \in E^*_\Pi \tag{10}$$

$$e_{p,q} + e_{q,r} - 1 \leq e_{p,r} \quad \forall (p, q, r) \in \Delta \tag{11}$$

These constraints read as follows: (9) ensures that if an action with precondition p achieves fact q, then p must precede q, (10) prevents cycles of length 2, meaning that if p precedes q, it cannot happen that q precedes p and (11) that if p precedes q and q precedes r, then p must precede r, for each triple (p, q, r) in Δ.

We refer to Feyzbakhsh Rankooh and Rintanen (2022) for the details and the proof of correctness of the resulting model, that we denote with VE(Π). The resulting model, while still polynomial, can be quite larger than TL(Π), as it requires $O(|P|^2)$ variables and $O(|A| \cdot |P|^2 + |P|^3)$ constraints. At the same time, its LP relaxation is provably stronger than the one of TL(Π).

4 B&C MIP formulations

Polynomial formulations, while appealing for their simplicity, are not necessarily the only approach for enforcing acyclicity in $G_{\hat{x}}$. Indeed, mathematical optimization has a decades-long experience in dealing with it in the context of the well-known TSP: numerous formulations have been proposed over the years, some of which polynomial, like those by Miller et al. (1960), Gavish and Graves (1978), Fox et al. (1980),

[2] In practice, a minimum degree heuristic is used to compute O.

Claus (1984), but the most effective one, dating back to Dantzig et al. (1954), yet still at the core of the state of the art TSP solver Concorde (Cook (2024)), is based on the so-called *subtour elimination constraint* (SEC) formulation and it has an exponential number of constraints. Of course, in this case we can no longer provide all the constraints upfront, but we need to be able to separate them dynamically during the solution process, in a B&C fashion (Padberg and Rinaldi (1991)). All modern MIP solvers provide explicit support for this scenario, through the availability of *lazy constraint* callbacks: each time an integer solution is found, either by some primal heuristic or at an integer node, a user defined callback is invoked, to check whether the solution can be accepted as a new incumbent, or rejected. In the latter case, one or more inequalities (the nominal lazy constraints) violated by the integer point can be returned to the solver.

4.1 Subtour elimination constraints

Our first attempt at using a B&C formulation is to exploit, with the due modifications, the class of subtour elimination constraints for delete-free planning as well. For any cycle C in the causal relation graph G_Π, we could impose the corresponding subtour elimination constraint $\sum_{(p,q)\in C} e_{p,q} \leq |C| - 1$, which forces the removal of (at least) one edge in the cycle. As anticipated, we cannot just add all those constraints upfront, as they are exponentially many. In addition, our basic model does *not* contain variables $e_{p,q}$, so we need to formulate the SEC in a different space.

Definition 4.1. Given a causal relation graph G_Π we define $\delta : P^2 \to A$ as a function mapping an edge (p, q) in G_Π to an arbitrary action $a \in A_{p\to q}$.

A SEC, due to the central role of first achievers in our base model IP(Π), has then the following structure:

$$\sum_{(p,q)\in C} z_{\delta(p,q),q} \leq |C| - 1 \tag{12}$$

We now provide the details of the separation procedure: given a point $\hat{x}$, consider the corresponding causal graph $G_{\hat{x}}$, and let $\delta(p, q)$ be the unique action $a \in A_{p\to q}$ such that $\hat{z}_{a,q} = 1$ (uniqueness is guaranteed by integrality and constraint (3)). It is easy to check whether $G_{\hat{x}}$ is acyclic and, if not, for every cycle C detected[3], return the violated SEC inequality. We denote the model that uses subtour elimination constraints as SEC(Π).

4.2 Disjunctive action landmarks

A completely different approach, that implicitly removes all cycles in $G_{\hat{x}}$, is based on the seemingly unrelated concept of landmarks. As anticipated in Section 2.2, the computation of the optimal plan for a delete-relaxed planning problem can be formulated as a minimum-cost hitting set problem over all of its landmarks. Given a complete set of landmarks F_L for a delete-free planning problem, a valid hitting set formulation thus reads:

$$\min \sum_{a\in A} \text{cost}(a) x_a \tag{13}$$

$$\sum_{a\in L} x_a \geq 1 \qquad \forall L \in F_L \tag{14}$$

$$x_a \in \{0, 1\} \qquad \forall a \in A \tag{15}$$

Note that this formulation does not enforce acyclicity explicitly, and does not even require variables y_p and $z_{a,q}$. This formulation is implicit in the work of Haslum et al. (2012), where the hitting set problem

[3] In practice we do not compute all cycles in the graph, but we limit ourselves to disjoint cycles: this can be done in linear time with a simple depth first search.

is solved (possibly multiple times) with a custom-made branch-and-bound procedure, with dual bounds coming from a combinatorial approximation algorithm and the linear programming relaxation of (13)-(15). We propose to use constraints (14) as cycle-breaking inequalities *on top* of our base model IP(Π): variables y_p, $z_{a,q}$ and constraints (3)-(7) are retained, providing additional LP bound tightening, while the entire optimization of the single branch-and-cut tree is delegated to the MIP solver. Our method can thus be seen as a more modern (and efficient) implementation of the approach in Haslum et al. (2012), where we exploit the power of modern MIP solvers and practices.

We now turn to the question on how to separate violated landmark constraints given an integer point $\hat{x}$.

It was shown by Bonet and Helmert (2010) that the set of landmarks that can be obtained as cut-sets of justification graphs (Section 2.3) is complete, meaning that they are sufficient to compute the value of h^+ via the formulation $(13) - (15)$. This, by itself, does not immediately give a polynomial separation procedure, as both the number of pcfs D and the number of cut-sets of G_D, for a given pcf, are exponential in the problem size. However, as shown by Bonet and Castillo (2011), the completeness proof by Bonet and Helmert (2010) readily gives a linear separation procedure, even though it was not framed as a separation problem. Let H be the set of actions used in a solution $\hat{x}$, and $R(H) \subseteq P$ be the maximal set of facts reachable from $\{s\}$ through a plan that only uses actions in H (note that $R(H) \neq S(H)$ if $\hat{x}$ does not provide a feasible solution). Then, if $t \notin R(H)$, a violated landmark can be obtained simply as:

$$L_H = \{a \in A \mid \text{pre}(a) \subseteq R(H), \text{add}(a) \not\subseteq R(H)\} \tag{16}$$

This corresponds to constructing a pcf D by picking any $p \in \text{pre}(a)$ for the actions a where $\text{pre}(a) \subseteq R(H)$, and $p \in \text{pre}(a) \setminus R(H)$ for the remaining ones. It is easy to show that H does not hit the s-t cut $(R(H), P \setminus R(H))$, and thus (16) gives a (maximally violated) landmark.

As observed by Haslum et al. (2012), the landmark obtained in this way is not necessarily minimal, so the corresponding cut could be dominated by other, equally violated, landmark constraints. In order to produce a minimal landmark, and thus a stronger cut, the same paper proposes a greedy heuristic that tries to obtain from H a maximal set of actions $H' \supseteq H$ such that the goal is still not reachable, i.e., $t \notin R(H')$: if H' is such a maximal set, then $A \setminus H'$ is a minimal landmark, still violated by $\hat{x}$. As noted by Haslum et al. (2012), with the appropriate data structures, the repeated reachability check implied by the greedy procedure outlined above can be done incrementally, and is quite efficient in practice. The separation procedure is formally described in Algorithm 1. The algorithm takes as input the planning problem and an integer feasible solution $\hat{x}$ of IP(Π). It starts by collecting in H the actions selected by $\hat{x}$ (line 2) and returns no cut if they already sufficient to reach the goal (line 3), as in that case $\hat{x}$ is a feasible plan. Otherwise the loop starting at line 5 greedily grows H with every action that does not make the goal reachable, so that at line 8 the set H is maximal among the non-reaching ones and its complement $A \setminus H$ is a minimal violated landmark. We denote the model that uses landmark constraints as LMC(Π).

Algorithm 1 LMC Separator for Integer Solutions

```
1: procedure SeparatorLMCInt(Π = (P, A, s, t, cost), x̂)
2:     H ← {a ∈ A | x̂_a = 1}
3:     if t ∈ R(H) then
4:         return {}        # The solution is feasible, no landmark to separate
5:     for a ∈ A \ H do
6:         if t ∉ R(H ∪ {a}) then
7:             H ← H ∪ {a}
8:     return A \ H
```

5 Warm-starting the MIP formulations

While experimenting with the MIP formulations described in the previous section, both old and new, we noticed a couple of common shortcomings. First, sometimes the MIP solver would struggle in finding a good feasible solution, or even just any, early on in the solution process: this was common to all formulations, but even more severe in the B&C formulations, where the incomplete nature of the model hinders the effectiveness of the primal heuristics implemented in the solver, a well-known side effect of lazy constraint generation. Then, we noticed that in some cases the LP relaxation at the root node was significantly worse than the value that could be computed, much more efficiently, by LM-cut (Section 2.4). This is particularly surprising for formulation LMC(Π): since the LM-cut heuristic eventually just computes a clever set of landmark constraints, formulation LMC(Π) has always at least the potential, depending on how often landmark constraints are separated and how they are chosen, to match the bound produced by LM-cut. Note, however, that in LMC(Π) we are only separating landmarks when integers solutions are found by the B&C process, something we have little control over. In order to have the guarantee of at least matching the LM-cut bound, we would need an exact separation procedure run without limits over fractional solutions, an approach that we address in Section 6. In this section we propose two (simple) techniques to fix those issues: a dedicated primal heuristic and the warmstarting of the model with LM-cut.

5.1 MIP start

Computing a plan for a delete-relaxed problem is trivial, since a simple dive, choosing at each step any applicable action, either finds a plan or proves the problem to be infeasible. Based on this observation, we designed a simple greedy heuristic that incrementally constructs a feasible solution, to be provided as a *MIP start* to the MIP solver. The method starts with the initial state $\{s\}$ and an empty sequence π and chooses, at each iteration, an applicable action a minimizing a given score, adding it to the sequence $\pi \leftarrow \pi + a$, until we reach the goal state $\{t\}$. After some preliminary experiments, we eventually settled on a lookahead strategy based on h^{add} (Section 2.1). This procedure is described in Algorithm 2. Each iteration of the loop starting at line 4 picks, among the actions applicable in the current state S, the one whose add effects minimize the h^{add} value of the resulting state (line 5), and applies it (line 7).

Algorithm 2 Primal Heuristic

```
1: procedure PrimalHeur(Π = (P, A, s, t, cost))
2:     S ← {s}        # Current state
3:     π ← []         # Plan as a list of actions
4:     while t ∉ S do
5:         a ← argmin_{a'} {h^add_{S∪add(a')} | a' ∈ {a | pre(a) ⊆ S}}
6:         π ← π + a
7:         S ← S ∪ add(a)
8:     return π
```

5.2 LM-cut initialization

Empirical studies have shown that h^{LM} is a very good approximation of h^+, often within a few percent. In other words, solving a hitting set problem over the set of landmarks computed by LM-cut is guaranteed to give at least h^{LM}, and thus any of the MIP formulations in this study can potentially benefit from those landmarks in terms of initial LP bound.

In our implementation we run LM-cut multiple times with different tie-breaking strategies, and collect all the produced landmarks; in particular, we execute LM-cut three times, using the arbitrary (ties don't change the pcf), inverse (ties always change the pcf) and value decrease minimization (prefer a precondition for which the $h^{\max}$ value decreased the least since the first iteration) policies (Lauer and Fickert (2020)). The runtime of LM-cut has always proved negligible w.r.t. the overall solution process.

6 Separating cuts on fractional solutions

In a modern branch-and-cut framework, it is very uncommon to separate lazy constraints only at integer solutions: it is, in general, more efficient to separate the same constraints also at fractional solutions, in particular at the root node, in order to improve the LP bound faster. This is a routine approach for both the TSP (Cook (2024)) and Benders decomposition (McDaniel and Devine (1977)), for example. Of course, given a family of inequalities, the separation procedure must be extended to deal with fractional solutions, and this sometimes becomes more expensive. For example, in the classical TSP case, separating SECs on integer solutions can be done in linear time with a simple graph search, while on fractional solutions this requires solving a sequence of max-flow problems: still polynomial, but more expensive both in theory and in practice.

In this section we describe separation procedures for separating SECs and landmarks over fractional solutions. We will see that the former can still be separated in polynomial time and prove that the latter is a NP-hard problem: for landmarks, we however provide an exact approach, based on a MIP formulation, and a polynomial separation heuristic. In the following, we denote with x^* a solution obtained by the relaxation LP(Π) of our base model.

6.1 SEC on fractional solutions

Let $G_{x^*} \coloneqq (P, E_\Pi, w)$ be the weighted causal relation graph with the weight function $w : P^2 \to [0, 1]$ defined as $w(p, q) \coloneqq (1 - z^*_{\delta(p,q),q})$ and $\delta(p, q) \coloneqq \text{argmax}_a\{z^*_{a,q}, \forall a \in A_{p\to q}\}$.

Theorem 6.1. *The fractional solution x^* violates a SEC (Equation 12) if and only if the graph G_{x^*} contains a directed cycle of total weight strictly less than* 1.

Proof. x^* violates a SEC iff there is a cycle C such that:

$$\sum_{(p,q)\in C} z^*_{\delta(p,q),q} > |C| - 1$$

By complementing the variables, this is equivalent to

$$\sum_{(p,q)\in C} (1 - z^*_{\delta(p,q),q}) < 1$$

By construction of G_{x^*}, this is equivalent to having a cycle of total weight ($\sum_{(p,q)\in C} w(p, q)$) less than 1. □

[4] This is analogous in spirit to the TSP fractional SEC separator (Padberg and Rinaldi (1991)), but different in two key ways: the underlying graph is directed, and we don't need to visit all nodes. The list of violated SEC can be computed in polynomial time by Algorithm 3. The algorithm enumerates at line 3 the candidate edges, i.e., those that can belong to a cycle of weight less than 1, and for each of them computes at

[4] In practice, for the fractional case, we use $\delta(p, q) = \text{argmax}_a\{z^*_{a,q}, \forall a \in A_{p\to q}\}$.

line 4 a shortest path closing the cycle, obtaining the shortest cycle through that edge; the test of line 5 then keeps the cycle only if its total weight is less than 1. The procedure thus performs $O(|E_\Pi|)$ shortest path computations on a graph with non-negative weights. [5]

Algorithm 3 SEC Separator for Fractional Solutions

```
1: procedure SeparatorSECFract(G_{x*} = (P, E_Π, w))
2:     cycles ← []        # list of violated cycles stored as list of edges
3:     for (p, q) ∈ E_Π with w(p, q) < 1 do
4:         S_{q,p} ← shortest_path(G_{x*}, from=q, to=p)
5:         if w(p, q) + Σ_{(u,v)∈S_{q,p}} w(u, v) < 1 then
6:             cycles ← cycles + [(p, q) + S_{q,p}]        # adding the violated cycle to the list of cycles
7:     return cycles
```

6.2 Landmarks on fractional solutions

As previously mentioned, deciding whether a fractional solution violates a landmark is an NP-hard problem. In this section we will focus on proving this result, while in the following we will first show an exact separator, based on a MIP formulation, and then a heuristic polynomial separator.

Remark 6.1. For simplicity, in this section, we will assume that each action has exactly one effect; this assumption is without loss of generality since given a problem Π, we can construct in linear time an equivalent problem that satisfies this assumption: $\Pi' = (P' = P \cup \{p_a, \forall a \in A\}, A' = \{(\text{pre}(a), \{p_a\}), \forall a \in A\} \cup \{(\{p_a\}, \{e\}), \forall a \in A, \forall e \in \text{add}(a)\}, s, t, \text{cost}')$, with the cost function being 0 for the new dummy actions linking new facts to each effect, and unchanged for all others. It is easy to see that the optimal solution of Π has the same cost as Π', and any plan of Π can be mapped to one of Π' and vice versa.

Lemma 6.1 (Bonet and Helmert (2010))**.** *For a given problem Π, a set of actions $L \subseteq A$ is a landmark if and only if there exists a pcf D for which L forms a cut-set in G_D.*

Since a landmark constraint is violated by x^* if and only if there exists a landmark L s.t. $\sum_{a \in L} x^*_a < 1$, Corollary 6.1 immediately follows:

Corollary 6.1. *A landmark is violated by x^* if and only if there exists a pcf D, for which there exists a cut-set L in the justification graph G_D, with $\sum_{a \in L} x^*_a < 1$.*

Due to the classical duality result between min-cut and max-flow:

Corollary 6.2. *A landmark is violated by x^* if and only if there exists a pcf D for which the max-flow of G_D, using the fractional value of actions x^*_a as capacities, is $f(G_D, x^*) < 1$.*

$$x^* \text{ violates a landmark } (\langle \Pi, x^* \rangle \in \text{VLM}) \iff \exists D : A \to P \text{ s.t. } f(G_D, x^*) < 1$$

Note that Corollary 6.2 is true due to our assumption of actions having only one effect, hence there are no multiple edges for the same action.

Theorem 6.2. *Given a delete-free planning problem Π and a fractional solution x^* obtained by* LP(Π)*, deciding whether x^* violates some landmark constraint is NP-hard.*

[5] In practice, as for the integer case, we only separate disjoint cycles.

Proof. To prove Theorem 6.2 we resort to a reduction from Vertex Cover (VC).

Given an undirected graph $G = (V, E)$, a *vertex cover* of G is a subset of the nodes $V' \subseteq V$ such that every edge of the graph has at least one of its two endpoints in V'. We consider here its decision version, which asks, given the graph G and an integer k, whether a vertex cover of size smaller than k exists. The reduction maps such a decision instance into a delete-free planning problem Π_{vc} and a fractional point x^*_{vc}, in such a way that the vertex covers of G correspond to the pcfs of Π_{vc}, and the size of a cover to the max-flow value of the associated justification graph.

Given a vertex cover problem $\langle G = (V, E), k\rangle$ we can construct a delete-free planning problem $\Pi_{\text{vc}} = (P_{\text{vc}}, A_{\text{vc}}, s, t, \text{cost}_{\text{vc}})$ as follows: the facts are the union between initial and goal facts, nodes and edges: $P_{\text{vc}} = \{s, t\} \cup V \cup \{e_{i,j} \mid (i, j) \in E\}$; we then have actions connecting the initial fact to the nodes $A_s = \{(\{s\}, \{v\}), \forall v \in V\}$, actions connecting nodes to edges $A_E = \{a_{ij} = (\{i, j\}, \{e_{i,j}\}), \forall (i, j) \in E\}$ and actions connecting edges to the goal fact $A_t = \{(\{e_{i,j}\}, \{t\}), \forall (i, j) \in E\}$: $A_{\text{vc}} = A_s \cup A_E \cup A_t$, each action with unit cost. Lastly, we construct the fractional solution: $x^*_{\text{vc},a} = \frac{1}{k}\ \forall a \in A_{\text{vc}}$. Note that this construction is linear both in time and space. The resulting instance is visualized in Fig. 1.

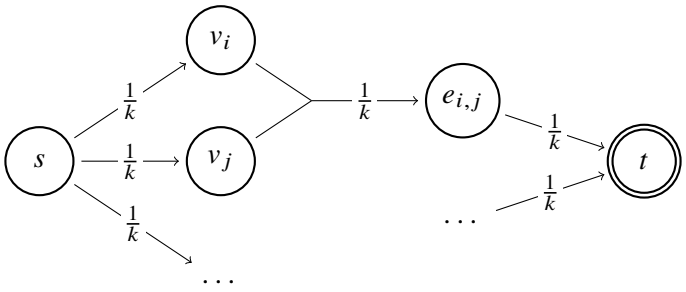


Figure 1: Problem Π_{vc} constructed from a VC instance.

The first part of the proof is proving $\langle G, k\rangle \in \text{VC} \Rightarrow \langle \Pi_{\text{vc}}, x^*_{\text{vc}}\rangle \in \text{VLM}$. Let us assume $\langle G, k\rangle \in \text{VC}$, with V' as a solution for it; we construct a pcf as follows: for each action in A_E, $D_{\text{vc}}(a_{ij}) = i$ if $i \in V'$ and j otherwise (if $j \in V'$), and the only precondition for all other actions. Since only nodes in V' have outgoing edges, and at most $\frac{1}{k}$ amount of flow reaches each of those nodes, $f(G_{D_{\text{vc}}}, x^*_{\text{vc}}) \leq \frac{|V'|}{k}$. Since we know that $|V'| < k$, due to our initial assumption, the max-flow on $G_{D_{\text{vc}}}$ is less than 1, hence $\langle \Pi_{\text{vc}}, x^*_{\text{vc}}\rangle \in \text{VLM}$. The second part is proving $\langle \Pi_{\text{vc}}, x^*_{\text{vc}}\rangle \in \text{VLM} \Rightarrow \langle G, k\rangle \in \text{VC}$. Let us assume $\langle \Pi_{\text{vc}}, x^*_{\text{vc}}\rangle \in \text{VLM}$ and D_{vc} be a pcf for which $f(G_{D_{\text{vc}}}, x^*_{\text{vc}}) < 1$. We can construct $V' = \{v \in V \mid \exists a \in A_E \text{ s.t. } D_{\text{vc}}(a) = v\}$. Since the only edge entering a fact e_{ij} in $G_{D_{\text{vc}}}$ is the one leaving its pcf $D_{\text{vc}}(a_{ij})$, every s-t-path has the form $s \to v \to e_{ij} \to t$ with $v = D_{\text{vc}}(a_{ij}) \in V'$, while the nodes outside V' are dead ends. Hence the edges $\{(s, v) \mid v \in V'\}$ form an s-t cut of capacity $\frac{|V'|}{k}$, and $f(G_{D_{\text{vc}}}, x^*_{\text{vc}}) \leq \frac{|V'|}{k}$. Conversely, let $a(v)$ be, for each $v \in V'$, an arbitrarily chosen action of A_E having v as chosen precondition. Since every action has a single precondition, the actions $\{a(v)\}_{v \in V'}$ are pairwise distinct, and so are their effect facts $e_{a(v)}$: the $|V'|$ paths $s \to v \to e_{a(v)} \to t$ are then pairwise edge-disjoint. As each of their edges has capacity $\frac{1}{k}$, routing $\frac{1}{k}$ units of flow along each of them is feasible, and $f(G_{D_{\text{vc}}}, x^*_{\text{vc}}) \geq \frac{|V'|}{k}$. The two bounds give $f(G_{D_{\text{vc}}}, x^*_{\text{vc}}) = \frac{|V'|}{k}$. By construction, V' covers all edges and, due to the flow being less than 1, $|V'| < k$. This means that $\langle G, k\rangle \in \text{VC}$. □

6.2.1 Exact algorithm

The exact algorithm relies on a MIP formulation based on the definition of disjunctive landmarks from justification graphs. We define binary variables z_a encoding whether a given action a is part of the cut-set

defining the landmark, and binary variables y_p encoding whether a fact p is on the s-side of the cut. Given a fractional solution x^*, the separation model reads:

$$\min \sum_{a \in A} x^*_a z_a \tag{17}$$

$$z_a \geq \sum_{p \in \text{pre}(a)} y_p - |\text{pre}(a)| + (1 - y_q) \qquad \forall a \in A, \forall q \in \text{add}(a) \tag{18}$$

$$y_s = 1 \tag{19}$$

$$y_t = 0 \tag{20}$$

$$z_a, y_p \in \{0, 1\} \qquad \forall a \in A, \forall p \in P \tag{21}$$

The objective (17) minimizes the left-hand side of the landmark w.r.t. the solution to separate x^* (i.e. maximizes the landmark violation), while constraints (18) impose that a can be part of the landmark if and only if all its preconditions are on the s side, but at least one of the effects is not. We are also imposing s to be on the s side (19), while forbidding t (20). In the following, we will denote with $\text{MC}(x^*)$ the optimal objective value of this separation MIP model.

Theorem 6.3. *Given a fractional solution x^*, it violates a landmark if and only if* $\text{MC}(x^*) < 1$.

Proof. By construction, given a solution z^* to $\text{MC}(x^*)$, $L = \{a \in A \mid z^*_a = 1\}$ is a landmark; moreover the set of landmarks computed in this way is complete due to Lemma 6.1. The optimal solution then returns *some* maximally violated landmark w.r.t. x^* and such landmark is violated if and only if $\sum_{a \in L} x^*_a < 1$ (or equivalently if $\text{MC}(x^*) < 1$). □

Note that the computation of $\text{MC}(x^*)$ can be interpreted as looking for the pcf D^* for which the max-flow on G_{D^*} is minimal, i.e., $\text{MC}(x^*) = \min_D f(G_D, x^*)$.

6.2.2 Heuristic approach using max-flow

As suggested by the proof of Theorem 6.2, given a fractional solution x^* and a pcf D, deciding whether a cut-set in G_D violated by x^* exists is as easy as a max-flow computation, which can be done in polynomial time. The hard part of our heuristic separator is then choosing the pcf D.

Before moving on to the choice of the pcf, we first need to address the fact that, unlike in the assumption for Theorem 6.2, we are dealing with actions that can have multiple effects: in our separator we transform the instance as described in Remark 6.1, setting $x^*_{a_d} = 1$ for all dummy actions $x^*_{a_d}$ connecting facts p_a and effects $\text{add}(a)$. We are now in a scenario in which each action has only one precondition (due to the chosen pcf) and one effect. The cut computed by any max-flow implementation will never include dummy actions, due to the original action's fractional value being a bottleneck for the flow going from the pcf to each of the expanded effects, so converting the computed landmark back to the original problem is a trivial transformation.

We now define two functions, similarly to $h^{\max}$ and h^{add} (Section 2.1), which we will use to choose our pcf:

$$f^{\max}_{x^*}(q) = \max_{a \in A_{\to q}} f^{\max}_{x^*}(a) \quad \forall q \in P \qquad f^{\max}_{x^*}(a) = \min\left\{ \min_{p \in \text{pre}(a)} f^{\max}_{x^*}(p), x^*_a \right\} \quad \forall a \in A \tag{22}$$

$$f^{\text{add}}_{x^*}(q) = \sum_{a \in A_{\to q}} f^{\text{add}}_{x^*}(a) \quad \forall q \in P \qquad f^{\text{add}}_{x^*}(a) = \min\left\{ \min_{p \in \text{pre}(a)} f^{\text{add}}_{x^*}(p), x^*_a \right\} \quad \forall a \in A \tag{23}$$

with $f^{\max}_{x^*}(s) = f^{\text{add}}_{x^*}(s) = 1$.

Lemma 6.2. *$f^{\max}_{x^*}(t) \leq f(G_D, x^*)$ for any D.*

Proof. Let D be an arbitrary pcf. We walk backwards, starting from t and ending in s, moving from fact q to $p = D(a(q))$, with $a(q) \in \operatorname{argmax}_{a \in A_{\to q}} f^{\max}_{x^*}(a)$; p is then the precondition, according to D, of an action attaining the maximum $f^{\max}_{x^*}(q)$. By definition $f^{\max}_{x^*}(p) \geq f^{\max}_{x^*}(a(q))$ and, by the choice of $a(q)$, $f^{\max}_{x^*}(a(q)) = f^{\max}_{x^*}(q)$; it follows that $f^{\max}_{x^*}(p) \geq f^{\max}_{x^*}(q)$. The arcs along the $s - t$ path just constructed have thus non-increasing $f^{\max}_{x^*}$ values and $f^{\max}_{x^*}(t)$ is the amount of flow that can be pushed through this path; this makes it a lower bound for the max-flow on G_D for any D, including any pcf D^* yielding $\mathrm{MC}(x^*)$.

□

Proposition 6.1. *Let $f^{\max}_{x^*}$, $f^{\mathrm{add}}_{x^*}$ and $\mathrm{MC}(x^*)$ be defined as in Equations* (17)*,* (22) *and* (23)*, then $f^{\max}_{x^*}(t) \leq \mathrm{MC}(x^*) \leq f^{\mathrm{add}}_{x^*}(t)$.*

Proof. $f^{\max}_{x^*}(t) \leq \mathrm{MC}(x^*)$: Lemma 6.2 holds for all pcfs, hence also for the one computed by $\mathrm{MC}(x^*)$.
$\mathrm{MC}(x^*) \leq f^{\mathrm{add}}_{x^*}(t)$: let D_{add} be any pcf that justifies the $f^{\mathrm{add}}_{x^*}$ values, i.e., such that

$$D_{\mathrm{add}}(a) \in \operatorname{argmin}_{p \in \mathrm{pre}(a)} f^{\mathrm{add}}_{x^*}(p).$$

In other words, we would compute the same exact $f^{\mathrm{add}}_{x^*}$ values starting from a simplified planning problem where each action has a single precondition, namely the one chosen by D_{add}. Since $\mathrm{MC}(x^*)$ is the minimum max-flow over all pcfs, $\mathrm{MC}(x^*) \leq f(G_{D_{\mathrm{add}}}, x^*)$, so it suffices to show that $f(G_{D_{\mathrm{add}}}, x^*) \leq f^{\mathrm{add}}_{x^*}(t)$. This is true because recursion (23) can be interpreted as a relaxation of a max-flow computation on the graph $G_{D_{\mathrm{add}}}$, where we no longer impose flow conservation at every node, but only require that the flow on each outgoing arc (p, q) is bounded by the total incoming flow at p, plus its own capacity x^*_a.

□

We can thus use those functions as guidelines to choose a pcf that likely returns a violated cut, if one exists. We then construct a pcf $D_{f\max/\mathrm{add}}$ by choosing as precondition of each action a fact with minimum $f^{\max}_{x^*}$, using $f^{\mathrm{add}}_{x^*}$ as tie-breaking, computing the max-flow with the resulting justification graph and returning a violated landmark whenever $f(G_{D_{f\max/\mathrm{add}}}, x^*) < 1$, otherwise we fail to return a cut. In addition, the proposition above also gives an inexpensive necessary condition for the existence of a violated landmark: since $f^{\max}_{x^*}(t) \leq \mathrm{MC}(x^*)$, whenever $f^{\max}_{x^*}(t) \geq 1$ we already know that no landmark constraint is violated by x^*, and the max-flow computation can be skipped altogether. The resulting separator is described in Algorithm 4: line 2 applies the transformation of Remark 6.1, so that each action of the justification graph has a single effect, and line 3 computes the two functions via a Dijkstra-like propagation. The test of line 4 implements the early-exit test just described, while the loop of line 6 builds the pcf. Line 9 finally computes a max-flow on the resulting justification graph, and its cut-set is returned as a violated landmark only if its capacity is below 1 (line 10), as prescribed by Corollary 6.2; the cut never contains dummy actions, so mapping it back to the original problem is immediate.

In our experimental evaluation, this separation heuristic managed to find a violated inequality in $> 97\%$ of the cases in which one existed, as certified by the exact MIP separation, while being, on average, one order of magnitude faster.

7 Computational results

We implemented our approaches in C++, and evaluated them on the standard International Planning Competition (IPC) benchmarks[6], for a total of 2799 planning problems. These instances are solved using a MIP

[6] https://github.com/AI-Planning/classical-domains

Algorithm 4 Heuristic LM Separator for Fractional Solutions

procedure SEPARATORLMFRACTHEUR($\Pi = (P, A, s, t, \text{cost}), x^*$)
  $(\Pi, x^*) \leftarrow$ single_effect(Π, x^*) # Remark 6.1, with $x^*_{a_d} = 1$ for the dummy actions
  compute $f^{\max}_{x^*}$ and $f^{\text{add}}_{x^*}$ # as defined in (22) and (23)
  if $f^{\max}_{x^*}(t) \geq 1$ **then**
    return {} # $f^{\max}_{x^*}(t) \leq \text{MC}(x^*)$, hence no landmark is violated
  for $a \in A$ **do**
    $P_a \leftarrow \text{argmin}_{p \in \text{pre}(a)} f^{\max}_{x^*}(p)$ # the preconditions of minimum $f^{\max}_{x^*}$
    $D(a) \leftarrow \text{argmin}_{p \in P_a} f^{\text{add}}_{x^*}(p)$ # ties are broken by $f^{\text{add}}_{x^*}$
  $(f, L) \leftarrow$ max_flow(G_D, x^*) # flow value and cut-set of a minimum s-t cut
  if $f < 1$ **then**
    return L # mapped back to the actions of the original problem
  return {} # the heuristic failed to find a violated landmark

solver, namely IBM ILOG CPLEX 22.1.0 (IBM ILOG (2022)), with default settings. The MIP solver was run on a cluster of 16 identical machines, each equipped with an Intel Xeon CPU E5-2623 V3 CPU running at 3.00 GHz, and 16 GB of RAM. Each method was run on each instance with a time limit of 15 minutes. The source code is available as a single solver on GitHub[7]: the results are measured on version 2.4.2 of said solver.

Before constructing a MIP model, each planning problem went through a preprocessing phase, consisting of landmark-based model reduction, relevance analysis and dominated actions elimination, as described by Imai and Fukunaga (2015), as well as forward relevance analysis (Haslum et al. (2012)). This preprocessing phase is common to all the formulations under test, and is well known to significantly speed up the solution of delete-free problems when encoded as MIP formulations.

In this section, the results are presented in tables comparing various models against a baseline. The baseline's absolute results appear on the left side of each table, while other methods are measured as ratios relative to the baseline (except for the number of instances solved, shown as deltas). Each row of the table corresponds to a subset of the instances, selected based on difficulty: in particular, the bracket $[l, u)$ collects all the instances where the shorter computing time among the methods under comparison is contained in the corresponding interval. Beyond the time brackets, three additional categories are shown: `all`, showing the results over all instances; `solvable`, selecting only the instances that could be solved to optimality by at least one method under comparison, and `all-solvable`, selecting only the instances that could be solved to optimality by all methods under comparison. Within each category, for time and nodes we report shifted geometric means (Achterberg (2007)), with a shift of 1.

For each comparison between the baseline and another method, we perform statistical tests on the results within each category to determine whether the difference is significant. We apply the Wilcoxon paired signed-rank test for nodes and times, using ratios of shifted values rather than differences to maintain consistency with the shifted geometric mean. For the number of instances solved, we use McNemar's test on the binary success status. When a method shows statistically significant differences from the baseline in a given category, i.e., the test returns a p-value < 0.05, its results are displayed in bold.

A summary of models considered in this work is described in Table 1: the first column lists all the models, the two central ones describe the constraints which define each of them and the latter the variants tested, obtained by adding the warm start techniques, indicated by the h and LM subscripts, respectively for the MIP start and the initial set of landmark constraints.

[7] https://github.com/Zanzibarr/MIPxHPLUS

Table 1: List of compared models

Model	Base Constraints	B&C Constraints	With Warm Start
TL(Π)	(2)-(8)	–	TL$_h$(Π), TL$_{h,\text{LM}}$(Π)
VE(Π)	(2)-(7) , (9)-(11)	–	VE$_h$(Π), VE$_{h,\text{LM}}$(Π)
SEC(Π)	(2)-(7)	(12)	SEC$_h$(Π), SEC$_{h,\text{LM}}$(Π)
LMC(Π)	(2)-(7)	(14)	LMC$_h$(Π), LMC$_{h,\text{LM}}$(Π)
(LM + SE)C(Π)	(2)-(7)	(12) , (14)	(LM + SE)C$_{h,\text{LM}}$(Π)
fSEC(Π)	(2)-(7)	(14) , Section 6.1	fSEC$_{h,\text{LM}}$(Π)
fLM$_{\text{MIP}}$(Π)	(2)-(7)	(14) , Section 6.2.1	fLM$_{\text{MIP},h,\text{LM}}$($\Pi$)
fLM(Π)	(2)-(7)	(14) , Section 6.2.2	fLM$_{h,\text{LM}}$(Π)
fLS(Π)	(2)-(7)	(14) , Sections 6.1 and 6.2.2	fLS$_{h,\text{LM}}$(Π)

7.1 Analysis of the current state of the art

In our first experiment, we compare the two known static models, namely TL(Π) and VE(Π), without any of the techniques proposed in this paper, in order to define a baseline for future comparisons. Aggregated results are given in Table 2. As expected from the literature, VE(Π) clearly outperforms TL(Π), by solving 165 more instances; moreover, given the stronger LP relaxation, VE(Π) requires only a fraction of the nodes and it is on average almost 40% faster. The improvement is also quite consistent across different brackets. Going forward, we use VE(Π) as the baseline for future comparisons.

Table 2: Comparison between state of the art models TL(Π) and VE(Π).

		TL(Π)			VE(Π)		
Category	Count	Solved	Nodes	Time (s)	Δ Solved	Nodes (rel.)	Time (rel.)
all	2799	2268	13.990	5.660	**+165**	**0.152**	**0.623**
solvable	2450	2268	9.093	2.364	**+165**	**0.131**	**0.513**
all-solvable	2251	2251	4.235	1.092	0	**0.198**	**0.686**
$[0, 1)$	1822	1812	1.863	0.295	**+10**	**0.133**	**0.483**
$[1, 10)$	354	292	180.612	18.515	**+62**	**0.021**	**0.213**
$[10, 100)$	188	129	583.999	115.768	**+49**	**0.025**	**0.261**
$[100, 900)$	134	35	401.762	712.435	**+44**	**0.074**	**0.418**

7.2 Effectiveness of B&C models

Next, we evaluate the effectiveness of B&C MIP formulations, based on subtour elimination (SEC(Π), described in Section 4.1) and landmark constraints (LMC(Π), described in Section 4.2). For the landmark model, we tested their separation both with and without the greedy strengthening procedure that makes the landmark minimal, and the former proved more effective, using 70% fewer nodes and $50 - 80$% less time on average. We thus enable this strengthening step in LMC(Π). We also tested both type of cuts, and we refer to this combination as (LM + SE)C(Π).

Table 3 shows that using subtour elimination constraints is not competitive with the current state of the art, resulting in significantly fewer models solved to optimality and increased runtimes across the board; on the other hand, using landmark constraints proves to be an improvement, reducing the computation time, with the only exception of the harder bracket. We note that the higher number of nodes explored is to be

Table 3: Comparisons between static and B&C models.

		VE(Π)			SEC(Π)			LMC(Π)		
Category	Count	Solved	Nodes	Time (s)	Δ Solved	Nodes (rel.)	Time (rel.)	Δ Solved	Nodes (rel.)	Time (rel.)
all	2799	2433	2.121	3.521	**-141**	**10.461**	**1.452**	+8	**3.173**	**0.936**
solvable	2517	2433	1.410	1.582	**-141**	**10.078**	**1.606**	+8	**2.820**	**0.882**
all-solvable	2217	2217	0.691	0.747	0	**6.637**	1.107	0	**2.628**	**0.766**
[0, 1)	1934	1934	0.450	0.248	**-33**	**8.174**	**2.000**	**-14**	**2.600**	**0.836**
[1, 10)	350	338	5.618	9.160	**-84**	**85.599**	**3.124**	-10	**7.116**	**0.801**
[10, 100)	156	130	16.675	69.634	**-29**	**48.309**	**1.561**	+9	**6.898**	0.795
[100, 900)	114	31	30.315	423.745	+5	**21.047**	**1.528**	**+23**	**4.559**	**1.257**

expected, given the fact that SEC(Π) and LMC(Π) are B&C models, and that constraints are added on the fly. We also tested the use of SEC on top of LMC(Π), but it did not provide a significant benefit (Table 4).

Table 4: Adding SEC to LMC(Π).

		LMC(Π)			(LM + SE)C(Π)		
Category	Count	Solved	Nodes	Time (s)	Δ Solved	Nodes (rel.)	Time (rel.)
all	2799	2444	6.424	3.214	+7	**1.160**	1.005
solvable	2490	2444	3.458	1.236	+7	1.069	1.008
all-solvable	2405	2405	2.565	0.865	0	1.004	1.045
[0, 1)	1902	1902	0.898	0.131	0	**0.860**	1.017
[1, 10)	327	323	26.199	4.685	0	1.212	1.243
[10, 100)	176	157	119.592	49.852	+9	1.503	**0.761**
[100, 900)	89	62	1046.660	415.373	-2	**4.721**	0.938

7.3 Warm-starting the MIP formulations

In this section we evaluate the effectiveness of the warm start techniques discussed in Section 5.1 and Section 5.2, namely the addition of a greedy primal heuristic to provide a MIP start to the solver and the addition of the landmark constraints generated by LM-cut as constraints to the formulation. For each of the formulations TL(Π), VE(Π), SEC(Π), LMC(Π), (LM + SE)C(Π), we consider the versions with the added MIP start, that we denote with h subscript, and then finally the versions where we also add the landmark constraints, that we denote with the h, LM subscript.

Table 5: Improvements on TL(Π).

		TL(Π)			$TL_h(\Pi)$			$TL_{h,LM}(\Pi)$		
Category	Count	Solved	Nodes	Time (s)	Δ Solved	Nodes (rel.)	Time (rel.)	Δ Solved	Nodes (rel.)	Time (rel.)
all	2799	2268	13.844	5.647	**+42**	**0.705**	**0.835**	**+122**	**0.266**	**0.636**
solvable	2396	2268	6.849	1.953	**+42**	**0.619**	**0.756**	**+122**	**0.182**	**0.471**
all-solvable	2257	2257	3.983	1.082	0	**0.701**	**0.787**	0	**0.242**	**0.565**
[0, 1)	1871	1863	2.033	0.381	-1	**0.675**	**0.747**	**+8**	**0.138**	**0.315**
[1, 10)	312	270	96.709	15.390	**+14**	**0.446**	**0.546**	**+42**	**0.063**	**0.232**
[10, 100)	156	104	374.789	128.544	**+28**	**0.180**	**0.509**	**+52**	**0.029**	**0.239**
[100, 900)	59	31	7812.194	550.706	+1	1.109	0.876	**+20**	**0.612**	**0.681**

Table 6: Improvements on VE(Π).

		VE(Π)			VE$_h$(Π)			VE$_{h,\mathrm{LM}}$(Π)		
Category	Count	Solved	Nodes	Time (s)	Δ Solved	Nodes (rel.)	Time (rel.)	Δ Solved	Nodes (rel.)	Time (rel.)
all	2799	2435	2.126	3.526	+14	**0.817**	**0.923**	**+61**	**0.389**	**0.793**
solvable	2505	2435	1.506	1.531	+14	**0.764**	**0.890**	**+61**	**0.308**	**0.704**
all-solvable	2415	2415	1.119	1.049	0	**0.791**	**0.907**	0	**0.347**	**0.758**
[0, 1)	1904	1904	0.604	0.210	0	**0.833**	**0.891**	0	**0.313**	**0.573**
[1, 10)	324	315	3.722	6.825	0	**0.600**	**0.802**	**+9**	**0.146**	**0.519**
[10, 100)	217	187	16.138	67.519	**+13**	**0.514**	**0.749**	**+29**	**0.150**	**0.503**
[100, 900)	225	29	9.229	392.234	+1	0.971	**0.902**	**+23**	**0.550**	**0.832**

Both TL(Π) and VE(Π) benefit from the proposed warm start techniques, with a significant improvement on every aspect (Tables 5 and 6). These results show a significant improvement while still using a ready-to-use MIP formulation, without the need of a B&C approach based on lazy constraints. We also note that both improvements contribute to the overall speedup, consistently across formulations and brackets.

Table 7: Comparison between state of the art models with warm start.

		TL$_{h,\mathrm{LM}}$(Π)			VE$_{h,\mathrm{LM}}$(Π)		
Category	Count	Solved	Nodes	Time (s)	Δ Solved	Nodes (rel.)	Time (rel.)
all	2799	2394	3.695	3.599	**+104**	**0.224**	**0.778**
solvable	2517	2394	2.541	1.557	**+104**	**0.176**	**0.714**
all-solvable	2375	2375	1.215	0.844	0	**0.250**	0.940
[0, 1)	1939	1931	0.764	0.220	**+8**	**0.222**	**0.665**
[1, 10)	334	285	20.793	11.339	**+44**	**0.031**	**0.422**
[10, 100)	180	146	34.848	60.264	**+26**	**0.068**	**0.633**
[100, 900)	235	32	73.069	629.846	**+26**	**0.114**	**0.477**

Comparing the static methods directly, we see that warmstarting did not change the relative ranking between TL(Π) and VE(Π), with the vertex elimination approach still clearly superior, see Table 7. We will thus use VE$_{h,\mathrm{LM}}$(Π) as the baseline for future comparisons.

Table 8: Improvements on SEC(Π).

		SEC(Π)			SEC$_h$(Π)			SEC$_{h,\mathrm{LM}}$(Π)		
Category	Count	Solved	Nodes	Time (s)	Δ Solved	Nodes (rel.)	Time (rel.)	Δ Solved	Nodes (rel.)	Time (rel.)
all	2799	2290	22.211	5.154	**+21**	**0.742**	**0.917**	**+120**	**0.253**	**0.651**
solvable	2421	2290	9.659	1.854	**+21**	**0.698**	**0.877**	**+120**	**0.191**	**0.493**
all-solvable	2278	2278	5.318	0.997	0	**0.711**	**0.870**	0	**0.263**	**0.652**
[0, 1)	1911	1880	2.658	0.423	-1	**0.836**	**0.896**	**+31**	**0.146**	**0.253**
[1, 10)	306	271	169.606	13.576	**+7**	**0.448**	**0.757**	**+35**	**0.059**	**0.298**
[10, 100)	141	107	2290.830	97.085	**+14**	**0.139**	**0.652**	**+33**	**0.037**	**0.385**
[100, 900)	67	32	13482.374	575.280	+1	**0.821**	0.943	**+21**	**0.506**	**0.633**

In Tables 8 and 9 we show the improvements on the B&C models as well. Again, both techniques give consistent improvements. Lastly, we reevaluate our B&C models against the baseline taking warm start techniques into account: while SEC$_{h,\mathrm{LM}}$(Π) is still not competitive with our baseline, LMC$_{h,\mathrm{LM}}$(Π) further

Table 9: Improvements on LMC(Π).

		LMC(Π)			LMC$_h$(Π)			LMC$_{h,\mathrm{LM}}$(Π)		
Category	Count	Solved	Nodes	Time (s)	Δ Solved	Nodes (rel.)	Time (rel.)	Δ Solved	Nodes (rel.)	Time (rel.)
all	2799	2442	6.660	3.288	**+22**	**0.732**	**0.923**	**+73**	**0.328**	**0.697**
solvable	2531	2442	3.871	1.465	**+22**	**0.684**	**0.891**	**+73**	**0.245**	**0.575**
all-solvable	2417	2417	2.621	0.894	0	**0.666**	**0.899**	0	**0.273**	**0.677**
[0, 1)	2014	1992	1.475	0.308	-5	**0.699**	**0.936**	**+22**	**0.175**	**0.353**
[1, 10)	328	315	30.438	9.845	**+7**	**0.626**	**0.761**	**+13**	**0.135**	**0.386**
[10, 100)	124	96	143.679	93.496	**+13**	**0.190**	**0.591**	**+26**	**0.050**	**0.367**
[100, 900)	262	39	800.856	441.708	+7	1.288	1.097	+12	0.925	0.796

widens the gap with VE$_{h,\mathrm{LM}}$(Π) by solving 22 more instances and reducing the computation time to below 70% of the baseline (Table 10).

Table 10: Comparison between static and B&C models with warmstarting.

		VE$_{h,\mathrm{LM}}$(Π)			SEC$_{h,\mathrm{LM}}$(Π)			LMC$_{h,\mathrm{LM}}$(Π)		
Category	Count	Solved	Nodes	Time (s)	Δ Solved	Nodes (rel.)	Time (rel.)	Δ Solved	Nodes (rel.)	Time (rel.)
all	2799	2498	0.826	2.804	**-84**	**6.871**	1.194	**+22**	**2.687**	**0.824**
solvable	2563	2498	0.502	1.344	**-84**	**7.528**	**1.239**	**+22**	**2.388**	**0.729**
all-solvable	2357	2357	0.270	0.792	0	**5.266**	**0.893**	0	**1.953**	**0.690**
[0, 1)	2017	2017	0.238	0.211	**-29**	**6.217**	1.648	**-11**	**1.635**	**0.722**
[1, 10)	339	326	0.502	9.257	**-48**	**56.004**	1.520	+9	**8.280**	**0.421**
[10, 100)	138	120	3.951	70.187	-4	**11.928**	**0.805**	**+11**	**4.113**	**0.583**
[100, 900)	200	35	10.397	363.968	-3	**21.489**	**1.725**	+13	**8.445**	**1.399**

We also compare LMC$_{h,\mathrm{LM}}$(Π) with (LM + SE)C$_{h,\mathrm{LM}}$(Π), to see whether the improvements made by the warm start techniques help the hybrid method, but as shown in Table 11, the use of SEC is still not justified by any significant improvement. Going forward, we will thus use LMC$_{h,\mathrm{LM}}$(Π) as the baseline for future comparisons.

Table 11: Adding SEC to LMC$_{h,\mathrm{LM}}$(Π).

		LMC$_{h,\mathrm{LM}}$(Π)			(LM + SE)C$_{h,\mathrm{LM}}$(Π)		
Category	Count	Solved	Nodes	Time (s)	Δ Solved	Nodes (rel.)	Time (rel.)
all	2799	2520	2.220	2.310	+7	**1.195**	**1.029**
solvable	2546	2520	1.073	0.900	+7	**1.255**	**1.046**
all-solvable	2501	2501	0.779	0.725	0	**1.280**	**1.090**
[0, 1)	2004	2002	0.271	0.111	+2	**1.299**	**1.044**
[1, 10)	333	330	4.264	3.626	+1	**1.742**	**1.430**
[10, 100)	147	137	21.465	42.407	+7	0.977	**0.689**
[100, 900)	62	51	354.147	325.437	-3	**1.871**	1.052

In Fig. 2 we analyze the dual gap at the end of the root node, i.e. the gap between the lower bound at the root node and the best-known primal solution, for the TL(Π), VE(Π) and LMC(Π) formulations, with and without warm start. Note that, in this plot, all instances for which the dual gap is 0 for all formulations under comparison have been removed, to highlight the differences on non-trivial instances. The plot confirms that

the TL(Π) model has a weaker LP relaxation than VE(Π), and shows that LMC(Π) sits in between the two.

At first sight this may look surprising, since LMC(Π) is an incomplete model, and one could expect its bound to be the weakest of the three. The two things are, however, not in contradiction: the quantity reported here is not the value of a fixed relaxation, but the bound at the *end* of the root node, i.e., the relaxation of the base model (2)-(7) tightened by all the landmark constraints (14) that the separator of Algorithm 1 happened to generate while processing the root. In other words, the position of LMC(Π) in the plot does not measure the strength of a formulation, but how many, and how strong, are the landmarks that the integer separation procedure manages to accumulate at the root: the relaxation of LMC(Π) without any separated constraint would indeed be the weakest of the three. What the plot shows is that the dynamic separation lands, on this benchmark, somewhere in between these two extremes. This is confirmed by measuring the root gap of $\text{fLM}_{\text{MIP},h,\text{LM}}(\Pi)$, i.e., $\text{LMC}_{h,\text{LM}}(\Pi)$ with the exact separation of landmark constraints on fractional solutions of Section 6.2.1: separating landmarks also at fractional points lowers the mean root gap over the same instances from 3.9% to 3.5%, below the 3.8% of $\text{VE}_{h,\text{LM}}(\Pi)$. Note that the cutloop retains the stopping criteria described below, so this is a conservative estimate: separating further violated landmark constraints can only strengthen the root relaxation. This also helps in reading the runtime results of Table 10: $\text{LMC}_{h,\text{LM}}(\Pi)$ explores about 2.7 times as many nodes as $\text{VE}_{h,\text{LM}}(\Pi)$, which is consistent with its weaker root bound, and yet it is faster and solves more instances, because of its faster node throughput. The trade-off between the strength of the bound and the cost of the LP relaxation is thus resolved, on these instances, in favour of $\text{LMC}_{h,\text{LM}}(\Pi)$.

We also note that warm-starting the formulations with the set of landmark constraints computed by LM-cut significantly reduces the dual gap across all formulations, bringing all medians down to 0, which is consistent with the explanation above: the LM-cut landmarks provide, upfront and at negligible cost, part of the constraints that the separator would otherwise have to discover during the cutloop.

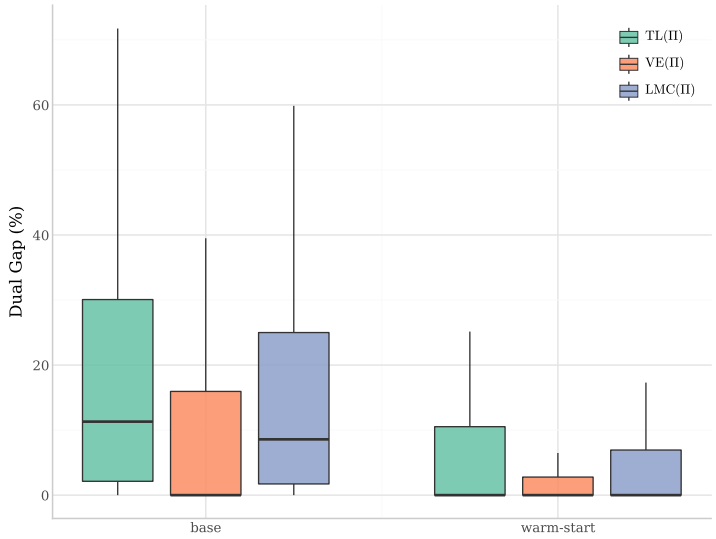


Figure 2: Dual gap at the end of the root node for each model.

7.4 Separating cuts on fractional solutions

In this section we enable the separation of violated constraints on fractional solutions, to see whether those can provide further improvements. In preliminary experiments, we found that, if implemented naïvely, the root cutloop with the separation of cuts at fractional solution could become way too expensive, with the vast

majority of the solution time spent in adding cuts with a very small effect on the LP bound. Due to this fact, and to have full control on the cutloop strategy, we implemented our own root cutloop, to be executed before solving the problem as a MIP with CPLEX. Our cutloop terminates when no violated cut has been found in the last iteration, if the relative MIP gap drops below 10% (w.r.t. the primal bound obtained by the solution constructed by our greedy heuristic) or if the improvement on the lower bound, with respect to the last 10 iterations, is below 0.5%. Other than at the root node, we separate cuts on fractional nodes as well, but only once per node. We evaluate the following models:

- $\text{fSEC}_{h,\text{LM}}(\Pi)$: $\text{LMC}_{h,\text{LM}}(\Pi)$ with violated subtour elimination constraints on fractional solutions, as described in Section 6.1;
- $\text{fLM}_{\text{MIP},h,\text{LM}}(\Pi)$: $\text{LMC}_{h,\text{LM}}(\Pi)$ with violated landmark constraints on fractional solutions, computed with the exact approach described in Section 6.2.1;
- $\text{fLM}_{h,\text{LM}}(\Pi)$: $\text{LMC}_{h,\text{LM}}(\Pi)$ with violated landmark constraints on fractional solutions, computed with the heuristic approach described in Section 6.2.2;
- $\text{fLS}_{h,\text{LM}}(\Pi)$: combination of $\text{fLM}_{h,\text{LM}}(\Pi)$ and $\text{fSEC}_{h,\text{LM}}(\Pi)$.

Table 12: Use of the MIP LM-based fractional separator.

		$\text{LMC}_{h,\text{LM}}(\Pi)$			$\text{fLM}_{\text{MIP},h,\text{LM}}(\Pi)$			$\text{fLM}_{h,\text{LM}}(\Pi)$		
Category	Count	Solved	Nodes	Time (s)	Δ Solved	Nodes (rel.)	Time (rel.)	Δ Solved	Nodes (rel.)	Time (rel.)
all	2799	2519	2.213	2.306	-2	**0.661**	**1.069**	+4	**0.793**	**1.027**
solvable	2541	2519	1.026	0.879	-2	**0.716**	**1.113**	+4	**0.868**	**1.045**
all-solvable	2495	2495	0.750	0.701	0	**0.726**	**1.142**	0	**0.896**	**1.071**
$[0,1)$	1998	1998	0.242	0.102	0	**0.735**	**1.309**	0	0.866	**1.094**
$[1,10)$	342	339	4.801	3.680	+3	**0.612**	**1.178**	+3	0.883	**1.092**
$[10,100)$	132	124	13.166	38.143	+3	**0.511**	1.047	+1	**0.748**	1.009
$[100,900)$	69	58	378.944	312.727	-8	**0.327**	**1.410**	0	**0.489**	1.163

As shown in Table 12, separating violated landmark constraints on fractional solutions manages to reduce the number of nodes explored; however, the total computation time is not yet competitive with our best model.

The heuristic separator manages to reduce the computation time at the expense of an increased number of nodes, due to the fact that it can fail to find a violated cut when one exists. Unfortunately, when comparing it with the baseline, it is still not competitive enough time-wise.

Finally, we evaluate the use of the SEC-based fractional-separator on our baseline but, once again, SEC do not seem to help significantly (Table 13).

7.5 Overall improvement

In this section, we measure the impact of all the methods proposed, comparing our best method $\text{LMC}_{h,\text{LM}}(\Pi)$ with the previous state of the art $\text{VE}(\Pi)$. Aggregated results are given in Table 14: overall, with the proposed model and warm-starting techniques, we manage to reduce to 70% the average number of nodes used to find the optimal solution, and to reduce the overall computing time by approximately 35%, and, more importantly, we can solve more instances to optimality within the time limit.

In Table 15, we show the comparison divided by problem domain: according to the table, our model outperforms the state of the art for most domains, but there are still some domains, like *barman, elevators, rovers*, in which the new method is not competitive with $\text{VE}(\Pi)$. There are also domains, like *petri* and

Table 13: Use of the SEC-based fractional separator.

		$\text{LMC}_{h,\text{LM}}(\Pi)$			$\text{fSEC}_{h,\text{LM}}(\Pi)$			$\text{fLS}_{h,\text{LM}}(\Pi)$		
Category	Count	Solved	Nodes	Time (s)	Δ Solved	Nodes (rel.)	Time (rel.)	Δ Solved	Nodes (rel.)	Time (rel.)
all	2799	2519	2.210	2.308	**+16**	0.992	**1.049**	+12	**0.692**	**1.071**
solvable	2561	2519	1.170	0.973	**+16**	1.013	**1.076**	+12	**0.755**	**1.110**
all-solvable	2485	2485	0.708	0.695	0	1.009	**1.104**	0	**0.758**	**1.142**
$[0, 1)$	2009	2008	0.282	0.110	+1	1.101	**1.107**	+1	**0.695**	**1.096**
$[1, 10)$	338	333	5.111	4.050	-3	0.968	**1.461**	-2	0.807	**1.628**
$[10, 100)$	145	129	19.944	50.938	**+11**	0.940	0.793	**+12**	**0.523**	0.802
$[100, 900)$	69	49	530.190	379.259	+7	0.878	0.994	+1	**0.329**	1.155

thoughtful, in which, even though we manage to beat the state of the art time-wise, we are exploring a much larger number of nodes, which might suggest that either the LP relaxation is still too weak or that our MIP start is not a good primal heuristic in those domains.

In Fig. 3, a scatter plot shows the per-instance improvements between $\text{VE}(\Pi)$ and $\text{LMC}_{h,\text{LM}}(\Pi)$, our best method, while in Fig. 4 we show the cumulative number of instances solved to optimality by $\text{VE}(\Pi)$, $\text{VE}_{h,\text{LM}}(\Pi)$, $\text{SEC}_{h,\text{LM}}(\Pi)$ and $\text{LMC}_{h,\text{LM}}(\Pi)$, showing that $\text{LMC}_{h,\text{LM}}(\Pi)$ manages to solve more instances within every time limit w.r.t. the other models.

Finally, in Fig. 4, we provide a survival plot comparing the state of the art $\text{VE}(\Pi)$ against its warm-started counterpart $\text{VE}_{h,\text{LM}}(\Pi)$ and our B&C methods $\text{SEC}_{h,\text{LM}}(\Pi)$ and $\text{LMC}_{h,\text{LM}}(\Pi)$. The plot confirms that even just warm-starting $\text{VE}(\Pi)$ has a positive effect, consistently increasing the number of instances that can be solved to optimality within each time limit, and the same applies in turn to our method $\text{LMC}_{h,\text{LM}}(\Pi)$. $\text{SEC}_{h,\text{LM}}(\Pi)$ can be considered somewhat competitive on smaller time limits, but it becomes less and less competitive as the instances become more difficult to solve.

Table 14: Comparison between state of the art and proposed model.

		$\text{VE}(\Pi)$			$\text{LMC}_{h,\text{LM}}(\Pi)$		
Category	Count	Solved	Nodes	Time (s)	Δ Solved	Nodes (rel.)	Time (rel.)
all	2799	2435	2.126	3.526	**+81**	1.045	**0.653**
solvable	2549	2435	1.510	1.773	**+81**	**0.747**	**0.523**
all-solvable	2402	2402	1.057	1.076	0	**0.736**	**0.501**
$[0, 1)$	2017	2013	0.704	0.342	-8	**0.548**	**0.459**
$[1, 10)$	334	306	4.460	17.966	**+26**	0.813	**0.205**
$[10, 100)$	143	96	30.462	111.187	**+37**	0.748	**0.396**
$[100, 900)$	208	20	8.981	401.510	**+26**	**4.724**	1.212

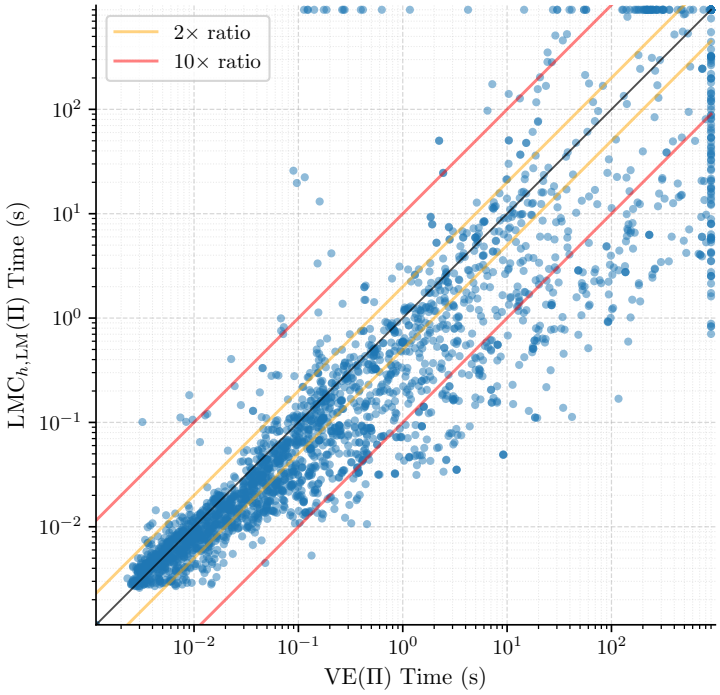


Figure 3: Scatter plot comparing the state of the art model VE(Π) against the new model $\mathrm{LMC}_{h,\mathrm{LM}}(\Pi)$.

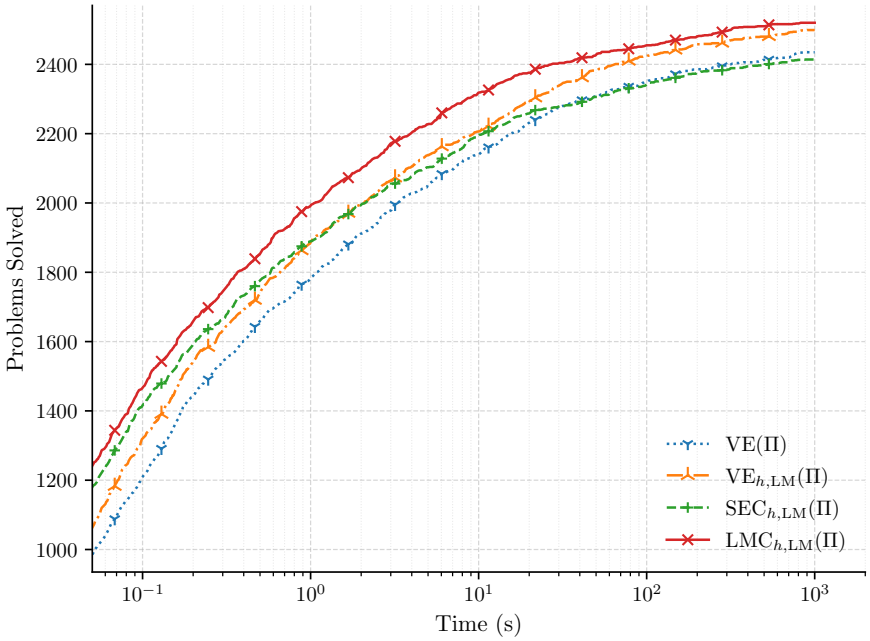


Figure 4: Cumulative number of instances solved by every model.

Table 15: Comparison between state of the art and proposed model, divided by domain.

		VE(Π)			$\text{LMC}_{h,\text{LM}}(\Pi)$		
Domain	Count	Solved	Nodes	Time (s)	Δ Solved	Nodes (rel.)	Time (rel.)
agricola	40	40	41.286	10.763	0	**0.033**	**0.213**
airport	50	50	3.429	0.395	0	**0.000**	**0.652**
barman	94	94	7.073	0.450	**-13**	1.361	4.744
blocks	35	35	0.000	0.006	0	1.000	0.900
childsnack	40	40	0.000	0.078	0	1.000	0.735
data	40	29	177.039	44.519	**+7**	**0.170**	**0.215**
depot	22	22	0.468	0.654	0	0.495	**0.337**
driverlog	20	19	6.497	2.842	0	0.738	0.874
elevators	100	100	0.452	0.855	-1	**128.460**	**2.430**
floortile	80	80	3.141	0.300	0	**0.014**	**0.324**
freecell	80	80	0.047	3.142	0	**65.547**	**0.390**
ged	40	40	0.000	3.293	0	1.000	1.077
grid	5	5	40.400	0.374	0	0.035	0.349
gripper	20	20	0.000	0.010	0	1.000	0.970
hiking	60	60	0.000	0.540	0	1.000	**0.875**
logistics00	28	28	0.000	0.009	0	1.000	0.672
logistics98	35	31	4.769	4.274	+4	**0.000**	**0.161**
miconic	150	150	0.000	0.012	0	1.000	0.699
movie	30	30	0.000	0.004	0	1.000	0.680
mprime	35	33	6.369	6.204	-1	**0.228**	**0.395**
mystery	30	30	2.198	1.763	-1	0.170	**0.511**
nomystery	40	40	0.000	2.638	0	1.000	**0.158**
openstacks	170	169	0.000	0.733	0	1.000	**0.889**
organic	52	27	0.619	45.339	**+7**	1.075	0.953
parcprinter	70	70	0.000	0.022	0	1.000	0.741
parking	80	80	0.064	1.587	0	**49.099**	**1.213**
pathways	30	30	0.000	0.130	0	1.000	1.155
pegsol	70	70	38.590	0.598	0	**0.011**	**0.052**
petri	20	20	0.149	9.982	0	**576.100**	**0.128**
pipesworld	100	47	6.565	79.699	+4	**18.597**	0.901
psr	50	50	0.000	0.005	0	1.000	0.815
rovers	40	40	0.000	0.170	0	1.000	**1.637**
satellite	36	36	0.000	1.770	0	1.000	**1.097**
scanalyzer	70	57	0.265	28.427	+5	**12.724**	**0.209**
schedule	150	149	0.000	0.008	0	1.000	0.798
snake	40	8	1.970	432.741	**+8**	**5.585**	0.375
sokoban	100	100	0.365	0.241	0	0.000	**0.125**
spider	40	18	5.529	143.843	**+20**	0.876	**0.107**
storage	30	30	0.000	0.361	0	1.000	**0.716**
termes	40	40	0.149	0.058	0	2.057	1.068
tetris	37	27	0.000	45.673	**+10**	1.000	**0.052**
thoughtful	40	5	0.653	553.137	0	**548.092**	0.793
tidybot	60	13	187.651	429.444	**+21**	**0.210**	**0.146**
tpp	30	30	3.053	1.585	0	**0.000**	**0.187**
transport	140	54	406.793	129.364	**-10**	**0.617**	0.971
trucks	30	30	0.000	0.100	0	1.000	0.833
visitall	80	59	35.402	18.907	**+21**	**0.008**	**0.144**
woodworking	100	100	0.000	0.280	0	1.000	0.904
zenotravel	20	20	0.177	0.538	0	0.000	**0.327**

8 Conclusions and future work

We analyzed existing MIP formulations for computing h^+, which consists of a base model plus a set of constraints required to model acyclicity in the causal relation graph associated to the problem Π, using timestamps and vertex elimination graphs. We then introduced different MIP formulations, which require an exponential number of constraints, and thus require the dynamic separation of violated constraints during the solution process: one formulation is based on SECs, while the other is based on disjunctive action landmarks. While the former did not prove competitive with the other methods, the latter managed to outperform the current state of the art. In addition, we proposed two warm-starting techniques, applicable to all MIP formulations, one providing a primal heuristic solution as a MIP start and the other initializing the set of constraints with the set of landmarks computed using the LM-cut heuristic. The warm-starting techniques proved effective in improving both the primal and dual behaviour of the MIP solution process, consistently across formulations. Finally, we tried separating cuts on fractional solutions, deploying a full branch-and-cut approach: this proved interesting on the theoretical side, as we could prove that the separation of SEC constraints over fractional solutions can be done in polynomial time, while it is NP-hard for landmark constraints. Unfortunately, with the current separation methods, we did not manage to obtain improvements in computing time, despite an encouraging reduction in the number of enumeration nodes. Combining the warm-starting techniques and the B&C model using landmarks, we managed to significantly improve over the current state of the art, reducing both the number of nodes and the time needed to compute h^+ with a MIP approach (Tables 14 and 15 and Figs. 3 and 4).

The present work leaves some interesting lines for future research: we found it quite surprising that cutting planes separated from fractional solutions did not prove effective in speeding up the solution process, despite being relatively inexpensive to separate: even though preliminary experiments with more sophisticated separation strategies based on in-out also proved ineffective, we think that other approaches, in particular improvements in the selection of violated cuts and a better tuning of the separation frequency, will eventually lead to an improved computation of h^+. In particular, the heuristic separator of Section 6.2.2 chooses the pcf from the primal fractional values alone, whereas LM-cut is driven by $h^{\max}$ on the *remaining* costs: a pcf driven by the LP reduced costs, or an LM-cut-style iterated cut-and-cost-reduction loop applied to them, looks like a natural way of obtaining stronger landmarks at a comparable cost.

9 Acknowledgments

We would like to express our sincere gratitude to Florian Pommerening, Malte Helmert, and Gabriele Röger for the many discussions (and patient explanations) on AI planning.

References

Tobias Achterberg. *Constraint Integer Programming*. PhD thesis, Technische Universität Berlin, 2007.

David L. Applegate, Robert E. Bixby, Vasek Chvátal, and William Cook. *The Traveling Salesman Problem: A Computational Study*. Princeton University Press, 2007.

Pierre Bonami, Domenico Salvagnin, and Andrea Tramontani. Implementing automatic benders decomposition in a modern mip solver. In Springer, editor, *IPCO 2020 Proceedings*, pages 78–90, 2020.

Blai Bonet and Julio Castillo. A complete algorithm for generating landmarks. In *Proceedings of the International Conference on Automated Planning and Scheduling*. AAAI Press, 2011. doi: 10.1609/icaps.v21i1.13482.

Blai Bonet and Héctor Geffner. Planning as heuristic search. *Artificial Intelligence*, 129, 2001. doi: 10.1016/S0004-3702(01)00108-4.

Blai Bonet and Malte Helmert. Strengthening landmark heuristics via hitting sets. In Helder Coelho, Rudi Studer, and Michael J. Wooldridge, editors, *ECAI*, volume 215 of *Frontiers in Artificial Intelligence and Applications*, pages 329–334. IOS Press, 2010. ISBN 978-1-60750-605-8.

Tom Bylander. The computational complexity of propositional strips planning. *Artificial Intelligence*, 69, 1994. doi: 10.1016/0004-3702(94)90081-7.

Nicos Christofides, Aristide Mingozzi, and Paolo Toth. State-space relaxation procedures for the computation of bounds to routing problems. *Networks*, 11:145–164, 1981.

A Claus. A new formulation for the travelling salesman problem. *SIAM Journal on Algebraic Discrete Methods*, 5(1): 21–25, 1984.

William Cook. Concorde TSP solver, 2024. `https://www.math.uwaterloo.ca/tsp/concorde.html`, Last accessed on June, 2024.

G. Dantzig, R. Fulkerson, and S. Johnson. Solution of a large-scale traveling-salesman problem. *Journal of the Operations Research Society of America*, 2(4):393–410, 1954. ISSN 00963984. URL `http://www.jstor.org/stable/166695`.

Toby O. Davies, Adrian R. Pearce, Peter J. Stuckey, and Nir Lipovetzky. Sequencing operator counts. In Subbarao Kambhampati, editor, *IJCAI*, pages 4140–4144, 2016. ISBN 978-1-57735-770-4 978-1-57735-771-1.

Masood Feyzbakhsh Rankooh and Jussi Rintanen. Efficient computation and informative estimation of h+ by integer and linear programming. *Proceedings of the International Conference on Automated Planning and Scheduling*, 32, 2022. doi: 10.1609/icaps.v32i1.19787.

Richard E. Fikes and Nils J. Nilsson. Strips: a new approach to the application of theorem proving to problem solving. *Artificial Intelligence*, 2:189–208, 1971.

Kenneth R Fox, Bezalel Gavish, and Stephen C Graves. An n-constraint formulation of the (time-dependent) traveling salesman problem. *Operations Research*, 28(4):1018–1021, 1980.

Bezalel Gavish and Stephen C Graves. The travelling salesman problem and related problems. *Massachusetts Institute of Technology, Operations Research Center Working Papers*, 1978.

Avitan Gefen and Ronen Brafman. The minimal seed set problem. *Proceedings of the International Conference on Automated Planning and Scheduling*, 21, 2011. doi: 10.1609/icaps.v21i1.13485.

Patrik Haslum. Incremental lower bounds for additive cost planning problems. *Proceedings of the International Conference on Automated Planning and Scheduling*, 22, 2012. doi: 10.1609/icaps.v22i1.13511.

Patrik Haslum, John Slaney, and Sylvie Thiebaux. Minimal landmarks for optimal delete-free planning. *ICAPS 2012 - Proceedings of the 22nd International Conference on Automated Planning and Scheduling*, 22, 2012. doi: 10.1609/icaps.v22i1.13534.

Malte Helmert and Carmel Domshlak. Landmarks, critical paths and abstractions: what's the difference anyway? In *Proceedings of the International Conference on Automated Planning and Scheduling*, 2009. doi: 10.1609/icaps.v19i1.13370.

IBM ILOG. CPLEX optimization studio 22.1. `https://www.ibm.com/products/ilog-cplex-optimization-studio`, 2022.

Tatsuya Imai and Alex Fukunaga. On a practical, integer-linear programming model for delete-free tasks and its use as a heuristic for cost-optimal planning. *Journal of Artificial Intelligence Research*, 54, 2015. doi: 10.1613/jair.4936.

Thorsten Klößner, Florian Pommerening, Thomas Keller, and Gabriele Röger. Cost partitioning heuristics for stochastic shortest path problems. In *ICAPS*, pages 193–202. AAAI Press, 2022.

Pascal Lauer and Maximiliam Fickert. Beating LM-cut with LM-cut: Quick cutting and practical tie breaking for the precondition choice function. In *ICAPS 2020 Workshop on Heuristics and Search for Domain-independent Planning*, 2020. URL `https://openreview.net/forum?id=anKt6unvU-`.

D. McDaniel and M. Devine. A modified Benders' partitioning algorithm for Mixed Integer Programming. *Management Science*, 4:312–319, 1977.

Clair E Miller, Albert W Tucker, and Richard A Zemlin. Integer programming formulation of traveling salesman problems. *Journal of the ACM (JACM)*, 7(4):326–329, 1960.

Manfred Padberg and Giovanni Rinaldi. A branch-and-cut algorithm for the resolution of large-scale symmetric traveling salesman problems. *SIAM Review*, 33(1):60–100, 1991.

Chiara Piacentini, Margarita P. Castro, André Augusto Ciré, and J. Christopher Beck. Compiling optimal numeric planning to mixed integer linear programming. In Mathijs de Weerdt, Sven Koenig, Gabriele Röger, and Matthijs T. J. Spaan, editors, *ICAPS*, pages 383–387. AAAI Press, 2018.

Florian Pommerening. *New Perspectives on Cost Partitioning for Optimal Classical Planning*. PhD thesis, Universität Basel, 2017.

Florian Pommerening, Gabriele Röger, Malte Helmert, Hadrien Cambazard, Louis-Martin Rousseau, and Domenico Salvagnin. Lagrangian decomposition for optimal cost partitioning. In *ICAPS*, pages 338–347, 2019.

Florian Pommerening, Thomas Keller, Valentina Halasi, Jendrik Seipp, Silvan Sievers, and Malte Helmert. Dantzig-wolfe decomposition for cost partitioning. In Susanne Biundo, Minh Do, Robert Goldman, Michael Katz, Qiang Yang 0001, and Hankz Hankui Zhuo, editors, *ICAPS*, pages 271–280. AAAI Press, 2021.

Nathan Robinson, Sheila McIlraith, and David Toman. Cost-based query optimization via ai planning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 28, 2014. doi: 10.1609/aaai.v28i1.9045.

Donald J. Rose, R. Endre Tarjan, and George S. Lueker. Algorithmic aspects of vertex elimination on graphs. *SIAM Journal on Computing*, 5(2):266–283, 1976. doi: 10.1137/0205021.

Jendrik Seipp and Malte Helmert. Counterexample-guided cartesian abstraction refinement for classical planning. *Journal of Artificial Intelligence Research*, 62:535–577, 2018.

Menkes Hector Louis van den Briel, Thomas Vossen, and Subbarao Kambhampati. Loosely coupled formulations for automated planning: An integer programming perspective. *Journal of Artificial Intelligence Research*, 31: 217–257, 2008.